

TJSC 2026 - Analista de Sistemas

Arquitetura de Sistemas e Integração

Aula 3 - Microsserviços, monólito, SOA, trade-offs e ambientes distribuídos

Disciplina	Arquitetura de Sistemas e Integração
Banca	FGV - Fundação Getulio Vargas
Cargo	Analista de Sistemas - TJSC 2026
Recorte da aula	Microsserviços; monólito; Service-Oriented Architecture - SOA; trade-offs; ambientes distribuídos
Objetivo	Converter vivência prática em Java/Quarkus, Angular, APIs e integrações em acerto de prova FGV.

Material autoral, orientado ao edital vigente e calibrado por referências técnicas primárias e por padrões de cobrança observados em provas recentes da FGV na área de Tecnologia da Informação. As questões são inéditas e não reproduzem itens reais.

1. Identificação da aula e recorte do edital

Esta é a Aula 3 da trilha de Arquitetura de Sistemas e Integração. Ela cobre o eixo conceitual que sustenta boa parte da disciplina: a comparação entre monólito, Service-Oriented Architecture - SOA, microsserviços e ambientes distribuídos. O edital menciona expressamente microsserviços, integração de sistemas e ambientes distribuídos e escaláveis; nesta aula, esses itens são tratados no limite necessário para decisão arquitetural e prova, sem transformar o estudo em tutorial de framework, mensageria, API Gateway ou Kubernetes, que aparecem em aulas próprias.

Item da trilha	O que será coberto aqui	O que fica para outras aulas
Microsserviços, monólito e SOA	Definições, diferenças, sinais de adoção correta, armadilhas e trade-offs.	Detalhes de Spring Cloud/Eureka/Zuul ficam para a Aula 6.
Ambientes distribuídos	Conceitos de rede, latência, falhas parciais, consistência, observabilidade, deployment e comunicação entre serviços.	Alta disponibilidade, resiliência e observabilidade em profundidade ficam para a Aula 10.
Integração de sistemas	Integração como motivação arquitetural; acoplamento, contratos e fronteiras.	REST, JSON, webhooks e APIs reversas ficam para a Aula 4.

A FGV costuma cobrar esse conteúdo por meio de cenário. A alternativa correta raramente é “microsserviços são melhores”. A banca tende a trocar causa por consequência: confunde implantação independente com mera separação de pacotes, confunde SOA com microsserviços, confunde distribuição física com bom desenho de domínio e trata escalabilidade como se fosse resultado automático de qualquer decomposição.

2. Termos essenciais da aula

Termo/sigla	Expansão ou definição operacional	Ponto de prova
API	Application Programming Interface - contrato pelo qual um componente expõe capacidades a outro.	API não implica microsserviço; monólitos também expõem APIs.
HTTP	Hypertext Transfer Protocol - protocolo de aplicação comum em integrações web.	Síncrono por natureza no uso request/response; não garante, sozinho, REST nem microsserviços.
SOA	Service-Oriented Architecture - arquitetura orientada a serviços.	Foco em capacidades expostas como serviços, contratos e integração corporativa.
Microservice	Serviço pequeno, autônomo, alinhado a capacidade de negócio e implantável de modo independente.	Serviço pequeno sem autonomia não basta.
Deployment independente	Capacidade de evoluir e publicar um serviço sem republicar todo o sistema.	É traço forte de microsserviços.
Ambiente distribuído	Sistema cujos componentes executam em processos, nós ou máquinas distintos e cooperam por rede.	Traz latência, falhas parciais e complexidade operacional.
Consistência eventual	Estado em que réplicas ou serviços convergem ao longo do tempo, sem leitura imediatamente consistente em todos os pontos.	É comum em integrações distribuídas, mas não é obrigatória em todo microsserviço.

3. Como a FGV pode cobrar este tema

Em provas recentes de Tecnologia da Informação, a FGV tem usado enunciados com órgão público, equipe de TI, migração de arquitetura, ambiente de nuvem, sistemas legados, microsserviços, mensageria, infraestrutura como código e integração. A estrutura típica é: um cenário de decisão técnica + alternativas com conceitos próximos. A questão separa quem conhece vocabulário de quem entende consequência arquitetural.

Sinais de estilo FGV para esta aula:

- Cenários institucionais: tribunal, órgão público, empresa estatal, plataforma corporativa, legado monolítico ou sistema de grande porte.
- Alternativas com termos verdadeiros em contexto errado: escalabilidade, acoplamento, orquestração, independência, mensageria, banco compartilhado, SOA e microsserviços.
- Cobrança de trade-off: a resposta correta reconhece benefício e custo; respostas absolutas tendem a ser suspeitas.
- Questões de exceção: “assinale a incorreta”, “não é característica”, “não decorre necessariamente”.
- Afirmativas I, II e III sobre autonomia, granularidade, implantação, dados e comunicação.

4. Monólito: conceito e leitura de prova

Monólito é uma aplicação construída, empacotada e implantada como uma unidade. Isso não significa código ruim. Um monólito pode ser modular, testável, bem organizado em camadas, com front-end separado e API limpa. O ponto arquitetural é que, em geral, a unidade de implantação é única: uma mudança relevante costuma exigir empacotar e publicar o conjunto da aplicação ou um grande módulo coeso.

No seu mundo Java/Quarkus, imagine uma aplicação com módulos de processos, usuários, documentos, pagamentos e notificações em um único repositório e um único artefato implantável. Ela pode ter packages, services, repositories, controllers/resources e banco normalizado. Ainda assim, se tudo sobe como uma aplicação única, a arquitetura continua monolítica do ponto de vista de deployment.

Aspecto	Monólito bem desenhado	Monólito problemático
Organização interna	Módulos claros, camadas, fronteiras internas e testes.	Código fortemente acoplado, regra espalhada, dependências circulares.
Deployment	Unidade única, mais simples de publicar e operar.	Toda pequena alteração pode exigir publicar o conjunto.
Dados	Transações ACID locais mais simples.	Banco vira centro de acoplamento entre áreas do sistema.
Operação	Menos componentes, menos rede interna, rastreamento mais simples.	Escala granular limitada; falha pode impactar grande parte da aplicação.
Quando faz sentido	Domínio ainda instável, equipe pequena, baixa necessidade de deploy independente.	Quando a complexidade de negócio cresce e o ciclo de mudança se torna lento.

Pegadinha: monólito não é sinônimo de aplicação sem camadas. Controller, Service e Repository indicam organização interna; não indicam, por si só, microsserviços.

5. SOA - Service-Oriented Architecture

SOA significa Service-Oriented Architecture, ou arquitetura orientada a serviços. O foco está em organizar capacidades de negócio como serviços acessíveis por consumidores, por meio de contratos, políticas e mecanismos de descoberta/interação. Em SOA corporativa, é comum encontrar barramentos de serviço, contratos mais centralizados, governança forte, integrações entre sistemas legados e serviços reutilizáveis por várias aplicações.

O modelo de referência da OASIS descreve SOA como um paradigma para organizar e utilizar capacidades distribuídas que podem estar sob diferentes domínios de propriedade. A ideia central para prova é: SOA resolve integração e reúso por serviços; não exige que cada serviço seja pequeno, independentemente implantável ou possua banco próprio.

Critério	SOA	Microsserviços
Foco original	Integração corporativa, contratos e capacidades compartilhadas.	Autonomia de serviços pequenos alinhados a capacidades de negócio.
Governança	Tende a ser mais centralizada, com padrões corporativos e barramento.	Tende a ser mais descentralizada, com autonomia por time/serviço dentro de guardrails.

Critério	SOA	Microserviços
Granularidade	Serviços podem ser mais grossos e reutilizáveis.	Serviços costumam ser menores e mais coesos.
Implantação	Nem sempre há deployment independente de cada capacidade.	Deployment independente é característica forte.
Dados	Pode haver bases compartilhadas e integração por sistemas corporativos.	Prefere dados descentralizados por serviço, quando viável.

Pegadinha: todo microserviço é “serviço” em sentido amplo, mas nem toda arquitetura orientada a serviços é arquitetura de microserviços. SOA pode usar SOAP, REST, mensageria e barramentos; microserviços podem usar REST, eventos ou gRPC. O protocolo não define sozinho a arquitetura.

6. Microserviços: definição com densidade de prova

Microserviços são um estilo arquitetural em que uma **aplicação é composta por um conjunto de serviços pequenos, executando em seus próprios processos**, comunicando-se por mecanismos leves e alinhados a capacidades de negócio. Martin Fowler e James Lewis destacam atributos como **serviços organizados em torno de capacidades de negócio, implantação independente, automação de deployment e gestão descentralizada de dados e tecnologia**.

A definição de prova não deve parar em “serviços pequenos”. O núcleo é a combinação de fronteira de negócio, autonomia, independência de implantação, comunicação por contrato e operação distribuída. Um sistema com dez endpoints em uma mesma aplicação não virou microserviços. Um monólito quebrado em vários serviços que compartilham o mesmo banco e precisam ser sempre publicados juntos também não atingiu a promessa principal.

Característica	O que significa	Armadilha FGV
Serviço alinhado ao negócio	A fronteira acompanha capacidade: processos, usuários, autenticação, documentos, pagamentos, notificações.	Dividir por camada técnica: serviço de Controller, serviço de Repository, serviço de DTO.
Baixo acoplamento	Mudanças internas não quebram consumidores quando o contrato é preservado.	Acoplamento oculto por banco compartilhado ou chamadas em cadeia.
Alta coesão	O serviço concentra responsabilidade bem delimitada.	Serviço “Deus” que centraliza várias áreas do domínio.
Deployment independente	Serviço pode ser publicado sem republicar todos os outros.	Pipeline único e release em bloco de todos os serviços.
Dados descentralizados	Cada serviço controla seus dados ou, no mínimo, sua fronteira transacional.	Vários serviços lendo e escrevendo as mesmas tabelas como se fossem módulos internos.
Automação operacional	CI/CD, monitoramento, logs, métricas, tracing e rollback planejados.	Criar muitos serviços sem maturidade operacional.

7. Monólito modular x microserviços

A FGV pode tentar induzir o candidato a achar que monólito é sempre errado. A resposta madura é: depende. Um monólito modular pode ser a melhor solução quando o domínio ainda muda muito, a equipe é pequena, o tráfego não exige escala independente por capacidade e a operação distribuída traria custo maior que benefício. Microserviços fazem mais sentido quando há necessidade real de deploy independente, escala granular, autonomia de times e fronteiras de negócio suficientemente estáveis.

Pergunta de arquitetura	Resposta que favorece monólito modular	Resposta que favorece microserviços
Quantos times atuam?	Poucos times, coordenação simples.	Múltiplos times autônomos por domínio/capacidade.
O domínio está estável?	Ainda há muita descoberta e mudança estrutural.	Fronteiras de domínio já são relativamente claras.
Preciso escalar tudo?	Carga homogênea; escalar a aplicação inteira é aceitável.	Carga assimétrica; uma capacidade exige escala diferente.

Pergunta de arquitetura	Resposta que favorece monólito modular	Resposta que favorece microsserviços
Como é o deployment?	Release menos frequente e bem coordenado.	Necessidade de releases independentes e frequentes.
Qual a maturidade operacional?	Observabilidade, automação e resposta a incidentes ainda limitadas.	CI/CD, tracing, monitoramento e governança já maduros.
Transações são simples?	Transação local ACID resolve a maioria dos casos.	Processos podem tolerar consistência eventual e compensações.

No contexto TJSC, uma aplicação de consulta administrativa com tráfego moderado e time único pode ser um monólito modular excelente. Já uma plataforma nacional com módulos independentes, vários tribunais consumidores, releases frequentes e serviços de integração pode justificar arquitetura distribuída e microsserviços. O erro de prova é tratar tecnologia como dogma.

8. Ambientes distribuídos: o preço da decomposição

Ambiente distribuído é aquele em que componentes executam em processos, máquinas, contêineres, nós ou regiões distintas e cooperam por rede. Microsserviços quase sempre criam um ambiente distribuído. Isso traz poder, mas também custos: latência, falhas parciais, timeouts, retries, versionamento de contratos, consistência de dados, observabilidade e segurança entre serviços.

Problema distribuído	Como aparece no sistema	Consequência de prova
Latência	Uma chamada local vira chamada de rede.	Quebrar demais pode piorar desempenho.
Falha parcial	Serviço A funciona, mas serviço B está indisponível.	Sistema precisa lidar com timeout, fallback e degradação.
Consistência	Cada serviço mantém parte do estado.	Transação única ACID entre serviços vira problema arquitetural.
Observabilidade	Erro atravessa vários serviços.	Logs isolados não bastam; métricas e rastreamento distribuído ajudam.
Versionamento	Consumidores dependem de contratos publicados.	Mudança de contrato deve ser compatível ou coordenada.
Segurança	Serviço chama serviço; identidade e autorização precisam trafegar.	TLS, tokens, políticas e auditoria ganham importância.

Pegadinha: distribuir componentes não torna o sistema automaticamente mais resiliente. Sem isolamento de falhas, timeouts, circuit breakers, observabilidade e automação, o sistema apenas troca complexidade local por complexidade de rede.

9. Comunicação entre componentes: síncrona e assíncrona

Embora REST, JSON, webhooks e RabbitMQ sejam aprofundados em aulas próprias, aqui é preciso entender o impacto arquitetural. Comunicação síncrona ocorre quando um componente espera resposta de outro para seguir. Em APIs HTTP, isso aparece como request/response. Comunicação assíncrona ocorre quando o emissor publica mensagem/evento e não precisa aguardar processamento imediato do consumidor. Em microsserviços, a escolha afeta acoplamento, latência, disponibilidade e consistência.

Critério	Comunicação síncrona	Comunicação assíncrona
Exemplo	API A chama API B e espera resposta.	Serviço publica evento PedidoCriado para consumidores.
Vantagem	Resposta imediata, fluxo simples de entender.	Desacoplamento temporal, maior tolerância a indisponibilidade temporária.
Custo	Propaga latência e falhas; pode criar cascata.	Mais difícil rastrear, testar e garantir ordem/consistência.
Quando usar	Consulta imediata, validação necessária em tempo real.	Notificação, processamento posterior, integração eventual.

Critério	Comunicação síncrona	Comunicação assíncrona
Pegadinha	Não é errado por si só.	Não elimina necessidade de contrato e tratamento de erro.

10. Dados e transações em microsserviços

O ponto mais perigoso em prova é achar que microsserviços são apenas “várias APIs”. A separação de dados é parte crítica. Em um **monólito, é comum uma única transação ACID alterar várias tabelas**. Em microsserviços, **cada serviço tende a controlar seus próprios dados**. Isso aumenta autonomia, mas dificulta consultas globais e transações distribuídas.

Em vez de uma transação única atravessando vários serviços, arquiteturas distribuídas costumam usar consistência eventual, eventos de domínio, **sagas** e transações compensatórias. O **padrão Saga, por exemplo, coordena uma sequência de transações locais e executa compensações quando uma etapa falha**. Isso reduz a dependência de transações distribuídas fortes, mas exige desenho cuidadoso.

Situação	Abordagem monolítica comum	Abordagem em microsserviços
Aprovar solicitação e registrar auditoria	Uma transação local no mesmo banco.	Evento SolicitaçãoAprovada consumido por serviço de auditoria.
Consulta que junta muitos dados	JOIN no banco central.	Composição por API, projeção materializada, CQRS ou leitura denormalizada.
Rollback de processo longo	ROLLBACK automático da transação.	Compensação explícita: cancelar reserva, desfazer lançamento, registrar estorno.
Banco compartilhado	Natural no monólito.	Sinal de acoplamento se vários serviços escrevem nas mesmas tabelas.

11. Estratégias de migração: do legado ao distribuído

A FGV pode contextualizar com modernização de sistemas legados. A resposta correta normalmente evita “reescrever tudo de uma vez”. Uma abordagem realista é estrangular gradualmente partes do monólito: identificar capacidades com alto valor ou alto acoplamento, criar contratos, extrair funcionalidades, redirecionar chamadas e manter coexistência temporária. Essa ideia é conhecida como Strangler Fig Pattern, embora a banca possa cobrar a lógica sem usar o nome.

Etapas	Decisão esperada	Erro comum
Diagnóstico	Mapear domínio, dependências, dados, tráfego e riscos.	Escolher tecnologia antes de entender fronteiras.
Modularização interna	Separar módulos e reduzir acoplamento dentro do monólito.	Extrair serviço com código ainda dependente de tudo.
Contrato	Definir API/eventos e compatibilidade.	Consumidores acessarem diretamente tabelas internas.
Extração gradual	Migrar uma capacidade por vez.	Big bang sem rollback seguro.
Operação	Adicionar observabilidade e automação antes de multiplicar serviços.	Criar muitos serviços sem logs, tracing, monitoramento e ownership.

12. Trade-offs que mais derrubam candidato

Tema	Benefício real	Custo real	Como vira pegadinha
Escalabilidade	Escalar só o serviço mais demandado.	Mais componentes, rede, orquestração e monitoramento.	Dizer que todo microsserviço escala melhor automaticamente.
Autonomia de times	Times podem evoluir serviços independentes.	Exige ownership, contratos, governança e disciplina.	Confundir autonomia com ausência de padrões.

Tema	Benefício real	Custo real	Como vira pegadinha
Tecnologia heterogênea	Escolher ferramenta adequada por serviço.	Aumenta custo de operação e suporte.	Tratar diversidade tecnológica como sempre desejável.
Banco por serviço	Reduz acoplamento e aumenta autonomia.	Dificulta consultas globais e transações.	Achar que cada serviço sempre precisa de SGBD diferente.
Comunicação remota	Integra componentes e permite distribuição.	Latência e falhas parciais.	Ignorar timeouts e retries.
Resiliência	Falha pode ser isolada.	Precisa desenho explícito de tolerância a falhas.	Achar que decompor já garante alta disponibilidade.

13. Síntese comparativa: monólito, SOA e microserviços

Critério	Monólito	SOA	Microserviços
Unidade de implantação	Aplicação única ou grande módulo.	Serviços corporativos; deployment pode variar.	Serviços independentes.
Foco	Simplicidade operacional e coesão interna.	Integração e reúso de capacidades.	Autonomia, escala granular e evolução independente.
Governança	Centralizada no time/sistema.	Mais centralizada e corporativa.	Descentralizada com padrões mínimos.
Dados	Banco único é comum.	Pode haver bases compartilhadas.	Dados preferencialmente controlados por serviço.
Risco principal	Acoplamento e releases lentos quando cresce.	Governança excessiva e barramento como gargalo.	Complexidade distribuída e operacional.
Boa resposta FGV	Pode ser adequado em contexto simples ou domínio instável.	Não é sinônimo de microserviço.	Não é solução universal; exige maturidade.

14. Pegadinhas FGV consolidadas

- “Quebrou em vários projetos, virou microserviço.” Errado. Pode ser apenas monólito distribuído ou módulos separados sem autonomia.
- “Cada endpoint é um microserviço.” Errado. Endpoint é contrato de acesso; microserviço é unidade arquitetural autônoma.
- “Microserviços dispensam governança.” Errado. Eles exigem contratos, observabilidade, segurança, versionamento e ownership.
- “SOA e microserviços são sinônimos.” Errado. Há interseção, mas SOA é mais ampla e frequentemente mais corporativa/centralizada.
- “Monólito é sempre antipadrão.” Errado. Monólito modular pode ser excelente e mais econômico.
- “Banco compartilhado entre serviços é sempre proibido.” Cuidado. É forte sinal de acoplamento, mas a prova pode exigir leitura contextual: legado, transição ou restrição institucional.
- “Distribuir melhora desempenho automaticamente.” Errado. Chamadas de rede podem piorar latência e disponibilidade percebida.
- “Consistência eventual significa inconsistência sem controle.” Errado. É modelo controlado de convergência, com contratos e compensações quando necessário.

15. Lista de questões autorais - padrão FGV

Resolva as 20 questões sem consultar o gabarito. As questões foram construídas para treinar distinções conceituais próximas, cenários arquiteturais, leitura de diagrama textual e trade-offs. Não reproduzem questões reais.

1. Uma equipe do TJSC mantém uma aplicação Java com módulos internos de usuários, processos, documentos e auditoria. A aplicação está bem organizada em camadas, possui APIs REST e usa um banco relacional único. Apesar disso, todo o sistema é empacotado e implantado como uma única unidade. Sob o ponto de vista arquitetural, a classificação mais adequada é:

- (A) arquitetura de microserviços, pois a existência de APIs REST caracteriza independência de implantação.
- (B) SOA pura, pois toda aplicação com serviços internos elimina a noção de monólito.
- (C) arquitetura orientada a eventos, pois a auditoria sempre pressupõe eventos assíncronos.
- (D) monólito modular, pois há organização interna, mas a unidade de implantação permanece única.
- (E) arquitetura peer-to-peer, pois os módulos internos possuem responsabilidades distintas.

2. Em relação a microserviços, assinale a afirmativa mais adequada.

- (A) São definidos principalmente pelo uso obrigatório de HTTP e JSON entre todos os componentes.
- (B) Devem ser alinhados a capacidades de negócio e possuir autonomia suficiente para evolução e implantação independente.
- (C) Correspondem a qualquer decomposição em classes, pacotes ou camadas dentro de uma aplicação Java.
- (D) Exigem que todos os serviços sejam escritos na mesma linguagem e compartilhem o mesmo banco de dados.
- (E) São indicados sempre que uma aplicação monolítica possui mais de três módulos funcionais.

3. Considere as afirmativas sobre monólito e microserviços. I. Um monólito pode ter boa modularização interna e ainda assim ser implantado como unidade única. II. Microserviços tendem a reduzir a complexidade operacional por eliminar chamadas de rede internas. III. Microserviços favorecem escala granular, mas introduzem desafios de consistência, observabilidade e versionamento. Está correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) I e II, apenas.
- (D) I e III, apenas.
- (E) I, II e III.

4. Um órgão público possui diversos sistemas legados que precisam interoperar. A área de arquitetura propõe expor capacidades de negócio como serviços reutilizáveis, com contratos e políticas corporativas, eventualmente mediadas por componentes de integração. A descrição aproxima-se mais de:

- (A) Domain-Driven Design, pois todo serviço corporativo é uma entidade de domínio.
- (B) Event sourcing, pois contratos de serviço exigem armazenamento imutável de eventos.
- (C) arquitetura cliente-servidor clássica, pois elimina a necessidade de contratos explícitos.
- (D) arquitetura MVC, pois separa visualização, controle e modelo em todos os sistemas.
- (E) Service-Oriented Architecture - SOA, por organizar capacidades como serviços em contexto de integração corporativa.

5. Em uma discussão de arquitetura, um analista afirma que a adoção de microserviços resolverá automaticamente problemas de disponibilidade, escalabilidade e manutenibilidade, sem necessidade de mudanças no processo de implantação ou operação. A crítica mais precisa é:

- (A) a afirmação é inadequada, pois microserviços exigem maturidade operacional, automação, observabilidade e tratamento de falhas distribuídas.
- (B) a afirmação é correta, pois microserviços eliminam a necessidade de monitoramento centralizado.
- (C) a afirmação é correta apenas quando todos os serviços usam o mesmo banco de dados.

- (D) a afirmação é inadequada porque microsserviços não podem ser usados em sistemas web.
- (E) a afirmação é correta se os serviços forem publicados dentro do mesmo artefato executável.

6. Um sistema foi dividido em cinco serviços. Todavia, qualquer mudança em um serviço exige recompilar e publicar todos os demais; além disso, todos acessam diretamente as mesmas tabelas de negócio. Essa situação é melhor descrita como:

- (A) microsserviços plenamente autônomos, pois há cinco processos diferentes.
- (B) arquitetura orientada a eventos, pois o banco compartilhado substitui mensageria.
- (C) decomposição distribuída com forte acoplamento, sem alcançar os benefícios esperados de microsserviços.
- (D) monólito modular ideal, pois serviços separados sempre aumentam coesão.
- (E) SOA obrigatoriamente correta, pois o compartilhamento de tabelas é requisito de todo serviço.

7. Em ambientes distribuídos, a ocorrência de falhas parciais significa que:

- (A) todos os componentes do sistema falham ao mesmo tempo, impossibilitando recuperação gradual.
- (B) apenas sistemas monolíticos podem ficar indisponíveis parcialmente.
- (C) a consistência eventual deixa de existir quando há falha de rede.
- (D) o balanceador de carga impede qualquer indisponibilidade de serviço.
- (E) alguns componentes podem continuar funcionando enquanto outros ficam indisponíveis ou lentos, exigindo timeouts, retries e degradação controlada.

8. Relacione os conceitos às descrições. 1. Monólito modular. 2. Microsserviço. 3. SOA. 4. Ambiente distribuído. () Organiza capacidades como serviços, com contratos e integração corporativa. () Componente autônomo, alinhado a capacidade de negócio e implantável independentemente. () Aplicação única bem organizada internamente em módulos. () Componentes executam em nós/processos distintos e cooperam por rede. A sequência correta é:

- (A) 2 - 3 - 1 - 4.
- (B) 3 - 2 - 1 - 4.
- (C) 1 - 2 - 4 - 3.
- (D) 4 - 1 - 3 - 2.
- (E) 3 - 1 - 2 - 4.

9. Sobre comunicação síncrona e assíncrona em arquiteturas distribuídas, assinale a afirmativa correta.

- (A) Comunicação assíncrona elimina a necessidade de contrato entre produtor e consumidor.
- (B) Comunicação síncrona é proibida em microsserviços por impedir qualquer escalabilidade.
- (C) Comunicação assíncrona garante processamento exatamente uma vez sem desenho adicional.
- (D) Comunicação síncrona tende a propagar latência e falhas, enquanto a assíncrona pode reduzir acoplamento temporal, com maior complexidade de rastreamento e consistência.
- (E) Comunicação assíncrona é sinônimo de SOA, e comunicação síncrona é sinônimo de monólito.

10. Um time quer extrair do monólito um módulo de notificações. O módulo recebe eventos de negócio, envia e-mails e mensagens, e pode evoluir com ciclo próprio. A decisão arquitetural mais coerente, em uma migração gradual, é:

- (A) extrair o módulo com contrato claro, eventos definidos, ownership e observabilidade, mantendo coexistência controlada com o monólito.
- (B) duplicar todas as tabelas do monólito no novo serviço, sem contrato de sincronização.
- (C) publicar o novo serviço somente se todos os demais módulos forem extraídos no mesmo dia.
- (D) permitir que qualquer serviço leia e escreva livremente as tabelas internas do módulo de notificações.
- (E) substituir a API por acesso direto do front-end ao banco de notificações.

11. Assinale a opção que apresenta uma característica que NÃO decorre necessariamente da adoção de microsserviços.

- (A) Possibilidade de implantação independente de serviços bem delimitados.
- (B) Maior necessidade de observabilidade em fluxos distribuídos.

- (C) Garantia automática de transações ACID globais entre todos os serviços.
- (D) Possibilidade de escalar capacidades específicas do sistema.
- (E) Necessidade de versionar contratos entre consumidores e provedores.

12. Em uma arquitetura de microsserviços, a recomendação de que cada serviço controle seus próprios dados tem como principal objetivo:

- (A) impedir o uso de bancos relacionais em qualquer serviço.
- (B) garantir que todos os serviços usem tecnologias de persistência diferentes.
- (C) eliminar a necessidade de auditoria e integridade de dados.
- (D) substituir regras de negócio por stored procedures compartilhadas.
- (E) reduzir acoplamento entre serviços e preservar autonomia de evolução, ainda que isso complique consultas e transações globais.

13. Leia o diagrama textual: Cliente Angular -> API de Processos -> Serviço de Documentos -> Serviço de Auditoria -> Banco de Auditoria. Em produção, a tela fica lenta porque a consulta de processos espera a auditoria síncrona concluir. A análise arquitetural mais adequada é:

- (A) a lentidão prova que microsserviços não podem usar bancos separados.
- (B) há uma cadeia síncrona que propaga latência; auditoria poderia ser tratada por evento assíncrono se a regra de negócio permitir.
- (C) a solução correta é unir todos os serviços no front-end Angular.
- (D) o problema decorre exclusivamente do uso de JSON.
- (E) o serviço de auditoria deve acessar diretamente o banco de processos para eliminar contratos.

14. Sobre Service-Oriented Architecture - SOA e microsserviços, analise as afirmativas. I. SOA pode envolver serviços mais grossos, governança corporativa e integração entre sistemas heterogêneos. II. Microsserviços enfatizam autonomia, implantação independente e fronteiras alinhadas ao negócio. III. SOA e microsserviços são sinônimos, diferindo apenas pelo nome usado pela ferramenta. Está correto o que se afirma em:

- (A) I e II, apenas.
- (B) I, apenas.
- (C) II, apenas.
- (D) I, II e III.
- (E) III, apenas.

15. Uma aplicação de pagamentos precisa executar, em uma única operação de negócio, reserva de saldo, emissão de comprovante, notificação e registro de auditoria. Em um monólito com banco único, a equipe usa transação local. Ao migrar para serviços autônomos, a alternativa conceitualmente mais adequada é considerar:

- (A) uma transação SQL única compartilhada por todos os bancos de todos os serviços, sem acoplamento.
- (B) acesso direto de todos os serviços à tabela de saldo para manter atomicidade.
- (C) a eliminação da auditoria, pois microsserviços não registram eventos globais.
- (D) sagas, eventos e compensações, quando a operação puder ser modelada como sequência de transações locais coordenadas.
- (E) substituição de todos os bancos por arquivos JSON em disco.

16. Em um projeto de modernização, a equipe decide começar reduzindo dependências internas do monólito, separando módulos por capacidade de negócio, definindo contratos e extraindo gradualmente funcionalidades para serviços externos. Essa estratégia é mais compatível com:

- (A) reescrita big bang sem coexistência entre arquiteturas.
- (B) migração incremental, frequentemente associada à ideia de estrangulamento gradual do legado.
- (C) normalização de banco de dados, que por si só transforma monólitos em microsserviços.
- (D) acoplamento por banco compartilhado como objetivo final de arquitetura.
- (E) substituição de camadas por classes utilitárias estáticas.

17. Considere as afirmativas sobre escalabilidade. I. Microsserviços podem permitir escala independente de capacidades específicas. II. Monólitos nunca podem ser escalados horizontalmente. III. A decomposição em serviços pode aumentar comunicação de rede e exigir observabilidade mais sofisticada. Está correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) I e III, apenas.
- (D) II e III, apenas.
- (E) I, II e III.

18. Em um ambiente de microsserviços, qual situação indica maior risco de acoplamento indevido entre serviços?

- (A) Cada serviço expõe contrato versionado para consumidores externos.
- (B) Serviços possuem pipelines de implantação próprios.
- (C) Logs e métricas são correlacionados por identificador de rastreamento.
- (D) Serviços publicam eventos de domínio para consumidores interessados.
- (E) Vários serviços escrevem diretamente nas mesmas tabelas internas para reaproveitar consultas existentes.

19. A respeito de consistência eventual, assinale a afirmativa correta.

- (A) Significa que o sistema não possui regra de negócio nem controle de integridade.
- (B) É o mesmo que inconsistência permanente causada por falha de modelagem.
- (C) É incompatível com qualquer arquitetura distribuída usada em sistemas corporativos.
- (D) Pode ser aceitável quando o domínio permite convergência posterior do estado, desde que eventos, compensações e regras de reconciliação sejam projetados.
- (E) Exige que todos os serviços compartilhem o mesmo banco de dados relacional.

20. Uma equipe afirma que adotou microsserviços porque criou um serviço para a camada de apresentação, outro para a camada de negócio e outro para a camada de persistência. Considerando boas práticas de decomposição, a avaliação mais adequada é:

- (A) a decomposição está correta, pois microsserviços devem corresponder obrigatoriamente às camadas MVC.
- (B) a decomposição está correta apenas se todos os serviços usarem o mesmo banco.
- (C) a decomposição é suspeita, pois microsserviços devem preferencialmente refletir capacidades de negócio, não camadas técnicas horizontais.
- (D) a decomposição caracteriza SOA e, por isso, dispensa contratos.
- (E) a decomposição elimina a necessidade de integração entre componentes.

16. Gabarito resumido

Questão	Gabarito
1	D
2	B
3	D
4	E
5	A
6	C
7	E
8	B
9	D
10	A
11	C
12	E
13	B
14	A
15	D
16	B
17	C
18	E
19	D
20	C

Distribuição orgânica observada após auditoria: A em 3 questões; B em 4; C em 5; D em 4; E em 4. A distribuição não foi usada como quota de elaboração; as alternativas foram reembaralhadas por plausibilidade e ausência de padrão visual óbvio.

17. Comentários completos

Questão 1 - Gabarito: D

A alternativa D é correta porque descreve uma aplicação modular internamente, mas implantada como unidade única. Isso é monólito modular. A alternativa A seduz porque APIs REST aparecem em microserviços, mas API REST não implica implantação independente. B confunde SOA com qualquer uso de serviços internos. C força auditoria como evento assíncrono obrigatório. E usa peer-to-peer fora de contexto: responsabilidades distintas em módulos internos não tornam os módulos pares autônomos de rede.

Questão 2 - Gabarito: B

B é correta: microserviços devem ser alinhados a capacidades de negócio e ter autonomia suficiente para evolução e deployment independente. A reduz arquitetura a protocolo/formato; HTTP e JSON são comuns, mas não definidores. C confunde decomposição de código com decomposição arquitetural. D erra porque microserviços não exigem mesma linguagem nem banco compartilhado. E absolutiza o uso: número de módulos não determina necessidade de microserviços.

Questão 3 - Gabarito: D

I está correta: monólito pode ser bem modular e continuar com deployment único. II está errada: microserviços aumentam, e não eliminam, desafios operacionais por chamadas de rede, observabilidade e falhas parciais. III está correta: escala granular vem junto de custos de consistência, versionamento e

rastreamento. Assim, D é o gabarito. A ignora III; B trata como correta uma afirmação falsa; C inclui II; E inclui tudo, inclusive II.

Questão 4 - Gabarito: E

E é correta porque o cenário descreve capacidades expostas como serviços, contratos, políticas e integração corporativa, típico de SOA. A mistura DDD com serviços corporativos; DDD não transforma serviço em entidade. B exige event sourcing sem base no enunciado. C fala em cliente-servidor, mas o ponto central é integração orientada a serviços. D confunde SOA com MVC, que é padrão de separação na camada de apresentação/aplicação.

Questão 5 - Gabarito: A

A é correta: microserviços não resolvem automaticamente disponibilidade, escalabilidade e manutenibilidade; exigem automação, observabilidade, contratos e tolerância a falhas. B é tentadora por prometer simplicidade, mas monitoramento fica mais necessário. C preserva banco compartilhado como se fosse virtude, quando tende a acoplar. D é falsa: microserviços podem ser usados em sistemas web. E contradiz a ideia de implantação independente.

Questão 6 - Gabarito: C

C é correta: há decomposição distribuída, mas com forte acoplamento de deployment e dados. A confunde quantidade de processos com autonomia. B inventa orientação a eventos sem eventos. D chama de monólito modular ideal algo que já foi distribuído de modo acoplado. E transforma banco compartilhado em requisito de SOA, o que não procede.

Questão 7 - Gabarito: E

E é correta: falha parcial é típica de sistemas distribuídos, nos quais alguns componentes seguem operando enquanto outros falham, ficam lentos ou inacessíveis. A descreve falha total. B é falsa porque falhas parciais são especialmente relevantes em distribuídos, não exclusivas de monólitos. C mistura consistência eventual com falha de rede sem relação necessária. D superestima balanceador: ele ajuda, mas não impede toda indisponibilidade.

Questão 8 - Gabarito: B

A sequência correta é 3 - 2 - 1 - 4. SOA organiza capacidades como serviços em integração corporativa. Microserviço é componente autônomo alinhado ao negócio. Monólito modular é aplicação única organizada internamente. Ambiente distribuído envolve execução em nós/processos distintos cooperando por rede. As demais sequências trocam foco de integração, autonomia e implantação.

Questão 9 - Gabarito: D

D é correta: comunicação síncrona propaga latência/falha; comunicação assíncrona reduz acoplamento temporal, mas aumenta complexidade de rastreamento, ordenação e consistência. A é falsa porque eventos também exigem contratos. B exagera: microserviços podem usar chamadas síncronas. C promete exactly-once sem desenho adicional, o que é armadilha clássica. E faz equivalências indevidas entre estilos de comunicação e arquiteturas.

Questão 10 - Gabarito: A

A é correta por combinar migração gradual, contrato, eventos, ownership e observabilidade. B duplica dados sem sincronização, criando inconsistência. C impõe big bang desnecessário. D viola encapsulamento de dados do serviço. E coloca front-end acessando banco, erro contra separação de responsabilidades e segurança.

Questão 11 - Gabarito: C

C é a alternativa que NÃO decorre de microserviços: transações ACID globais automáticas entre serviços não são garantidas; pelo contrário, tornam-se difíceis. A, B, D e E são consequências ou necessidades plausíveis: implantação independente, observabilidade, escala granular e versionamento de contratos. A

tentação está em confundir microserviços com banco único transacional.

Questão 12 - Gabarito: E

E é correta: controlar os próprios dados reduz acoplamento e preserva autonomia, ainda que complique consultas globais e transações. A é falsa: bancos relacionais podem ser usados. B exagera: tecnologias diferentes são possíveis, não obrigatórias. C elimina indevidamente auditoria/integridade. D centraliza lógica em stored procedures compartilhadas, aumentando acoplamento.

Questão 13 - Gabarito: B

B é correta: o diagrama mostra cadeia síncrona, e auditoria pode ser evento assíncrono se o domínio permitir. A conclui indevidamente que bancos separados são o problema. C joga regra para o front-end, piorando arquitetura. D culpa JSON, que é apenas formato. E propõe acesso direto ao banco de processos, elevando acoplamento.

Questão 14 - Gabarito: A

I e II estão corretas: SOA pode ser mais corporativa e governada; microserviços enfatizam autonomia, fronteiras de negócio e deployment independente. III está errada porque SOA e microserviços não são sinônimos. B e C deixam uma correta de fora. D inclui III. E considera correta apenas a afirmação falsa.

Questão 15 - Gabarito: D

D é correta: em serviços autônomos, operações que antes eram transações locais podem exigir sagas, eventos e compensações. A promete transação SQL global sem acoplamento, irrealista. B usa tabela compartilhada como solução, acoplando serviços. C elimina auditoria sem justificativa. E substitui persistência séria por arquivo JSON, distrator técnico inadequado.

Questão 16 - Gabarito: B

B é correta: a descrição indica migração incremental e estrangulamento gradual do legado. A é o oposto, uma reescrita big bang. C confunde normalização com arquitetura de serviços. D torna acoplamento por banco objetivo final, o que é ruim. E troca modularização por utilitários estáticos, antipadrão que não resolve fronteiras.

Questão 17 - Gabarito: C

I está correta: microserviços podem escalar capacidades específicas. II está errada: monólitos podem ser escalados horizontalmente, embora com menor granularidade. III está correta: decomposição aumenta comunicação de rede e demanda observabilidade. Logo, C. A ignora III; B e D incluem II como verdadeira; E aceita tudo.

Questão 18 - Gabarito: E

E é correta porque vários serviços escrevendo nas mesmas tabelas internas criam acoplamento forte de dados e dificultam evolução independente. A, B, C e D são sinais positivos ou compatíveis com arquitetura distribuída madura: contratos versionados, pipelines próprios, correlação de logs e eventos de domínio. A pegadinha é confundir reuso de banco com integração saudável.

Questão 19 - Gabarito: D

D é correta: consistência eventual pode ser aceitável quando o domínio permite convergência posterior e há desenho de eventos, compensações e reconciliação. A e B tratam como falta de controle, o que é falso. C diz que é incompatível com sistemas corporativos, mas muitos domínios aceitam esse modelo. E volta ao banco compartilhado, que não define consistência eventual.

Questão 20 - Gabarito: C

C é correta: decompor por camadas técnicas horizontais tende a gerar chamadas remotas entre apresentação, negócio e persistência, com alto acoplamento e baixa coesão de negócio. A confunde

microsserviços com MVC/camadas. B preserva banco único como requisito indevido. D diz que SOA dispensa contratos, quando contratos são centrais. E ignora que serviços precisam integrar-se por APIs, eventos ou contratos.

18. Referências consultadas

- TJSC - Edital nº 10/2026, retificado em 10/04/2026 - conteúdo programático de Arquitetura de Sistemas e Integração.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- FGV Conhecimento - cadernos e editais públicos de concursos de Tecnologia da Informação usados apenas para calibrar estilo, densidade e formato de cobrança, sem reprodução literal de questões.
- Martin Fowler e James Lewis - Microservices: definição, características e trade-offs do estilo arquitetural.
- Microsoft Azure Architecture Center - Microservices architecture style; Saga design pattern; cloud design patterns.
- OASIS - Reference Model for Service Oriented Architecture 1.0.
- AWS - Monolithic vs microservices architecture; visão comparativa de arquitetura monolítica e microsserviços.

Observação: este material é autoral e voltado à preparação para prova. Não substitui a leitura do edital nem a resolução de questões reais da banca.